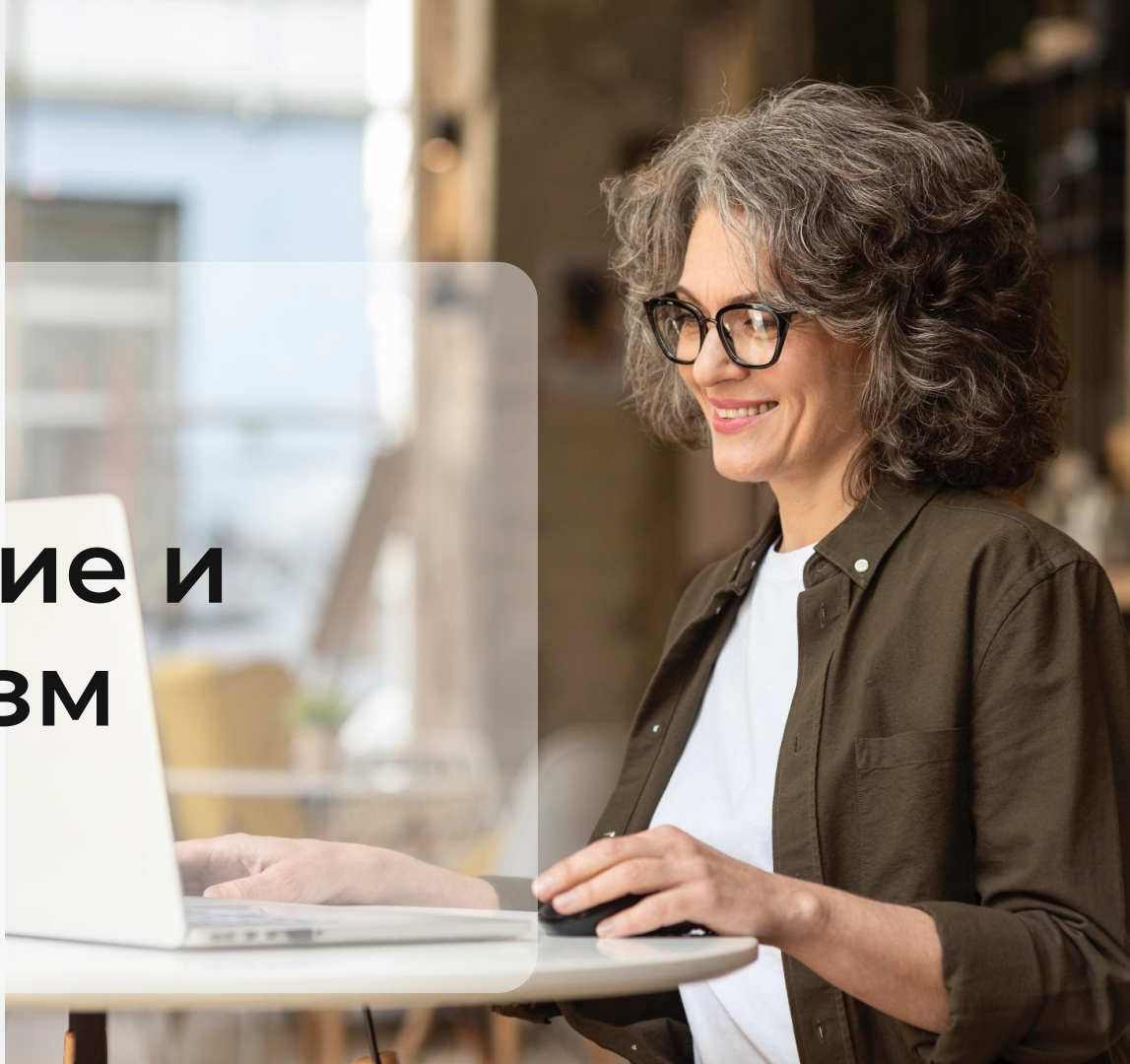


Python

Наследование и полиморфизм



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение



ООП



ООП vs функциональный подход



Создание класса



Метод `__init__`



Поля, методы и атрибуты



`self`



Создание объекта



Доступ к полям объекта



Методы класса

План занятия

- Принципы ООП
- Наследование
- Полиморфизм
- Функция super
- Функции isinstance и isinstance
- Наследование от object



ОСНОВНОЙ БЛОК





Принципы ООП

Принципы ООП

Наследование

Один класс может **унаследовать** свойства и поведение другого. Это позволяет **избегать дублирования** кода и **переиспользовать логику**

Инкапсуляция

Объект **скрывает внутреннее устройство** и предоставляет **только нужный интерфейс**.
Позволяет защищать данные и контролировать доступ к ним

Полиморфизм

Одинаковый интерфейс может **вести себя по-разному** в зависимости от типа объекта. Позволяет использовать **одинаковый код для разных классов**

Абстракция

Выделение **общих характеристик и поведения**, чтобы задать интерфейс для будущих реализаций в наследниках

Знание принципов



Помогает понять, как правильно проектировать классы и объекты



Упрощают понимание механизмов наследования, переопределения и взаимодействия объектов



Позволяют писать гибкий, расширяемый и поддерживаемый код



ВОПРОСЫ





Наследование



Наследование

Это механизм, который позволяет одному классу перенять свойства и поведение другого. Это один из ключевых принципов ООП, который помогает избегать дублирования кода и повторно использовать общую логику.

Наследование

Родительский класс, базовый класс или
суперкласс

Класс, от которого наследуются

Дочерний класс, производный класс или
подкласс

Класс, который наследует

Польза наследования



Избежание дублирования кода, путем выделения общего поведения в один базовый класс



Удобство расширять существующую логику без переписывания старых классов



Работа с разными объектами одинаковым образом, если они наследуются от общего предка

Наследование: синтаксис

```
class Parent:
    # родительский класс
    ...

class Child(Parent):
    # дочерний класс, наследует всё от Parent
    ...
```

Наследование



Пример

```
class Employee:
    def __init__(self, name):
        self.name = name

    def work(self):
        print(f"{self.name} is working...")

class Programmer(Employee):
    pass

class Manager(Employee):
    pass

programmer = Programmer("Alice")
manager = Manager("Bob")

programmer.work()
manager.work()
```

Пояснение

- Employee — базовый класс, который хранит имя сотрудника и умеет работать.
- Programmer и Manager **наследуют всё от Employee**: и поле name, и метод work().
- Каждый из них может использовать эти возможности **без повторного определения**.

Особенности



Дочерний класс наследует все поля и методы родительского класса



Можно добавлять свои методы и поля, не затрагивая родителя



Можно переопределять поведение, если нужно изменить работу унаследованного метода



ВОПРОСЫ





Полиморфизм



Полиморфизм

(от греч. «много форм»)

Это возможность использовать один и тот же интерфейс для разных типов объектов.

Польза полиморфизма



Позволяет писать универсальный код, не заботясь о типе объекта



Упрощает расширение программы — можно добавлять новые типы, не меняя старую логику



Делает код гибким и читаемым

Механизмы полиморфизма



Полиморфизм в Python реализуется через переопределение методов (overriding), когда дочерний класс заново определяет метод, унаследованный от родителя. При вызове такого метода используется реализация из дочернего класса.

Пример: переопределение метода

```
class Employee:

    def __init__(self, name):
        self.name = name

    def work(self):
        print(f"{self.name} is working...")

class Programmer(Employee):
    def work(self):
        print(f"{self.name} writing code...")

class Manager(Employee):
    def work(self):
        print(f"{self.name} managing team...")

staff = [Programmer("Alice"), Manager("Bob"), Programmer("Bill")]

for person in staff:
    person.work() # Поведение зависит от конкретного типа
```

- У всех сотрудников есть метод `work()`, но он реализован по-разному.
- В списке `staff` мы храним объекты разных классов: `Programmer`, `Manager`.
- При вызове `person.work()` Python **автоматически вызывает нужную версию метода** в зависимости от того, к какому классу относится объект.
- Это и есть **полиморфизм: один вызов → разное поведение в зависимости от типа объекта**.



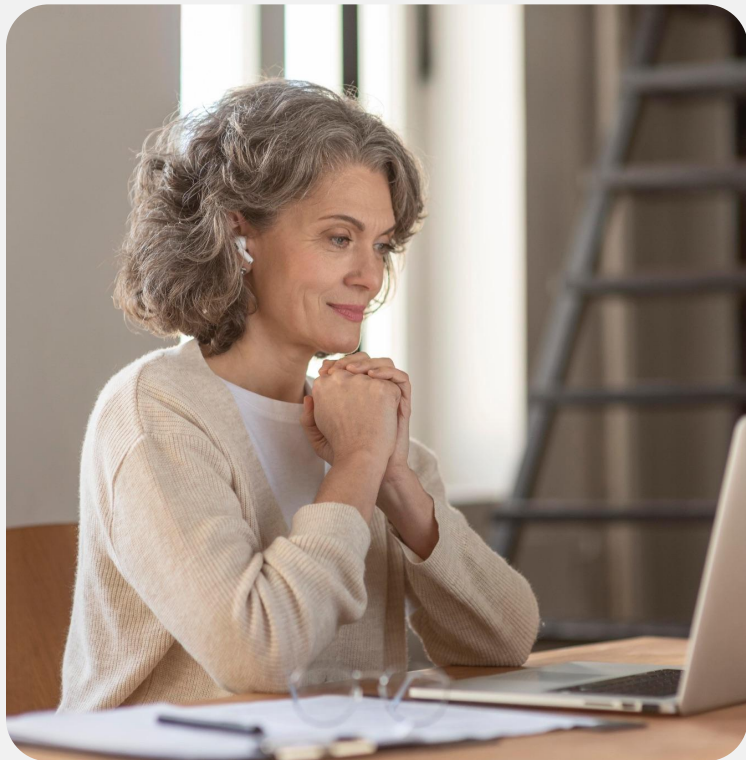
ВОПРОСЫ





ЗАДАНИЯ





Выберите верный вариант ответа

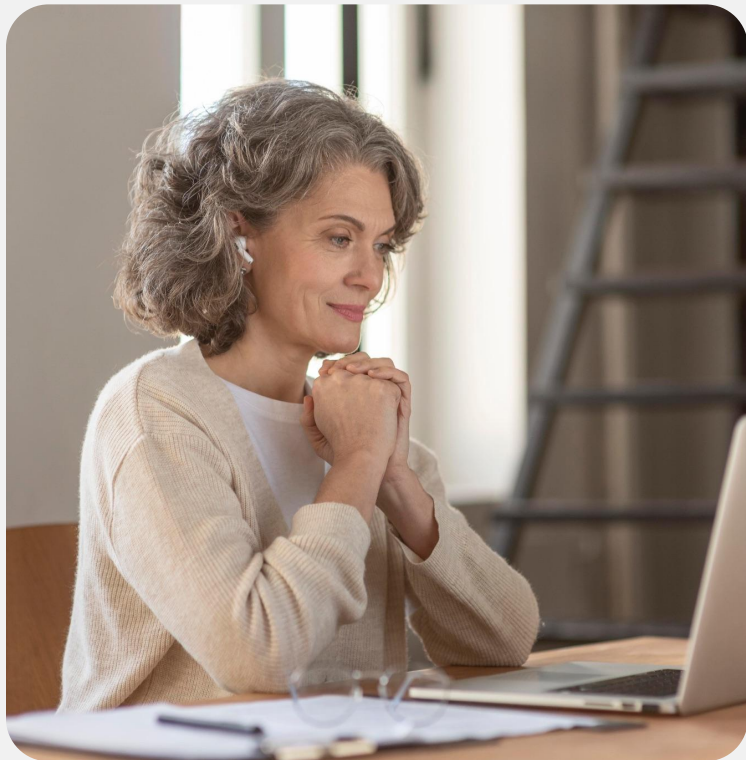
1. Что произойдёт при выполнении следующего кода?

```
class Employee:
    def work(self):
        print("Employee is working")

class Manager(Employee):
    pass

person = Manager()
person.work()
```

- a. Возникнет ошибка, так как Manager не имеет метода work()
- b. Будет напечатано: Employee is working
- c. Будет напечатано: None
- d. Метод work() выполнится, но ничего не выведет



Выберите верный вариант ответа

1. Что произойдёт при выполнении следующего кода?

```
class Employee:
    def work(self):
        print("Employee is working")

class Manager(Employee):
    pass

person = Manager()
person.work()
```

- a. Возникнет ошибка, так как Manager не имеет метода work()
- b. Будет напечатано: Employee is working
- c. Будет напечатано: None
- d. Метод work() выполнится, но ничего не выведет



Выберите верные варианты ответа

2. Укажи, что относится к преимуществам наследования:

- a. Повышение безопасности данных
- b. Возможность писать меньше кода
- c. Возможность скрыть поля от внешнего доступа
- d. Возможность переиспользовать логику



Выберите верные варианты ответа

2. Укажи, что относится к преимуществам наследования:

- a. Повышение безопасности данных
- b. Возможность писать меньше кода
- c. Возможность скрыть поля от внешнего доступа
- d. Возможность переиспользовать логику



ВОПРОСЫ





Функция super

Важно



При наследовании часто возникает ситуация, когда родительский класс уже задаёт общие поля в `__init__()`, а в дочернем классе необходимо добавить новые поля. Чтобы **не дублировать одинаковый код** используется функция `super()` — она вызывает ближайший метод родителя.

Пример

```
class Employee:
    def __init__(self, name):
        self.name = name

class Programmer(Employee):
    def __init__(self, name, language):
        super().__init__(name) # вызываем родительский __init__
        self.language = language

class Manager(Employee):
    def __init__(self, name, department):
        super().__init__(name) # вызываем родительский __init__
        self.department = department

p = Programmer("Alice", "Python")
print(p.name)
print(p.language)
```

Функция `super` в обычных методах



Функция `super` полезна не только в `__init__()`, но и **в любых других методах**, где нужно **расширить поведение родителя**, а не полностью его заменить.

Пример: расширение обычного метода

```
class Employee:
    def work(self):
        print("Employee is doing general tasks.")

class Programmer(Employee):
    def work(self):
        super().work() # вызываем метод родителя
        print("Programmer is writing code.")

class Manager(Employee):
    def work(self):
        super().work()
        print("Manager is holding a meeting.")

staff = [Programmer(), Manager()]
for person in staff:
    person.work()
    print()
```

Эффективность super



Автоматически находит метод в ближайшем родителе



Позволяет избежать дублирования кода



Гарантирует корректную работу при изменении иерархии классов

Использование super



Когда **родитель** уже делает нужную инициализацию, и мы хотим её сохранить



Когда **расширяем** (а не полностью заменяем) поведение метода



Особенно важно при **множественном наследовании** — `super()` автоматически учитывает порядок вызова



ВОПРОСЫ





ЗАДАНИЯ



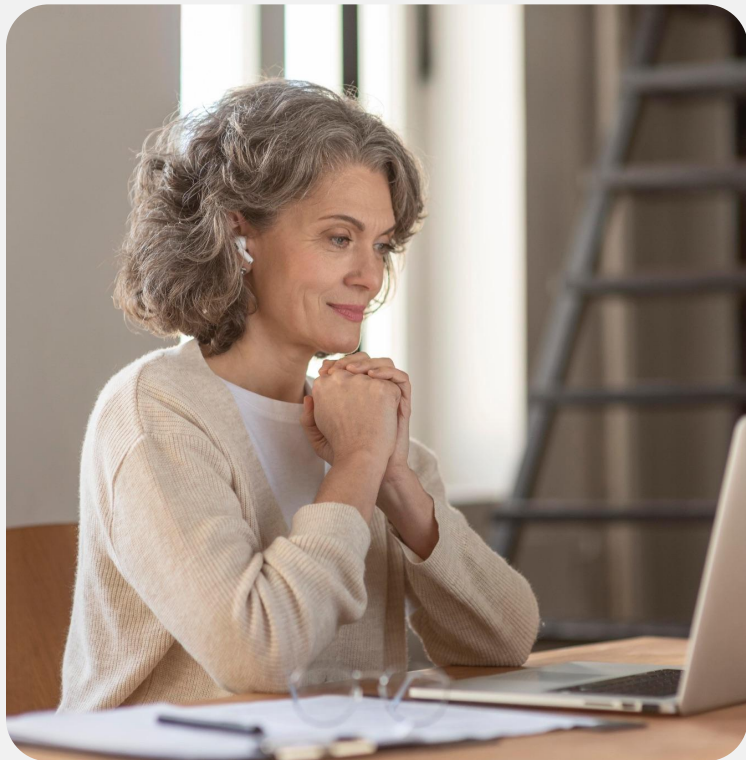


Выберите верные варианты ответа

1. Найди ошибку в коде:

```
class Employee:
    def __init__(self, name):
        self.name = name

class Programmer(Employee):
    def __init__(self, name, language):
        super(name).__init__()
        self.language = language
```

Выберите верные варианты ответа

1. Найди ошибку в коде:

```
class Employee:
    def __init__(self, name):
        self.name = name

class Programmer(Employee):
    def __init__(self, name, language):
        super(name).__init__()
        self.language = language
```

Ответ: Неверное использование `super`: должно быть `super().__init__(name)`



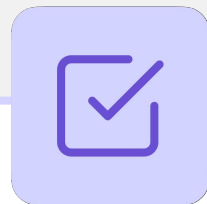
ВОПРОСЫ





Функции `isinstance` и
`issubclass`

Важно



Python предоставляет удобные встроенные функции для **проверки типов и иерархии классов**, что особенно полезно при работе с **наследованием** и **полиморфизмом**.



isinstance

Это встроенная функция Python, которая позволяет проверить, является ли объект экземпляром определённого класса или его подкласса, чтобы выбрать правильное поведение

Особенности isinstance



Если объект принадлежит **указанному классу или его наследнику**, функция возвращает True, в противном случае — False.

Функция `isinstance` также работает со **встроенными типами** (`str`, `list`, `int` и т.д.)

Функция isinstance



Синтаксис

```
isinstance(obj, some_class)
isinstance(obj, tuple_of_classes)
```

Пояснение

- `obj` — объект, который необходимо проверить
- `some_class` — класс, принадлежность к которому необходимо проверить
- `tuple_of_classes` — кортеж из нескольких классов, принадлежность к любому из которых необходимо проверить

Пример

```
class Employee:
    pass

class Programmer(Employee):
    pass

class Manager(Employee):
    pass

e = Employee()
p = Programmer()
m = Manager()

print(isinstance(p, Programmer)) # экземпляр класса
print(isinstance(p, Employee))  # экземпляр наследника
print(isinstance(p, Manager))   # Manager не находится выше в иерархии
print(isinstance(p, object))    # True – все классы наследуют от object
```


Пример использования в коде

```
class Employee:
    def work(self):
        print("Выполняет общие задачи")

class Programmer(Employee):
    def write_code(self):
        print("Пишет код")

class BackendDeveloper(Programmer):
    def write_code(self):
        print("Пишет серверный код")

class FrontendDeveloper(Programmer):
    def write_code(self):
        print("Пишет интерфейс")
```

1

```
class Manager(Employee):
    def work(self):
        print("Проводит собрание")

staff = [
    Programmer(),
    BackendDeveloper(),
    FrontendDeveloper(),
    Manager(),
    Employee()
]

for person in staff:
    if isinstance(person, Programmer):
        person.write_code()
    else:
        person.work()
```

2

Проверка на несколько классов



Пример

```
print(isinstance("hello", (str, int))) #  
Строка принадлежит к одному из классов
```

Пояснение

Можно проверить, принадлежит ли объект **к одному из нескольких классов**, передав кортеж



issubclass

Это встроенная функция Python, которая позволяет проверить, является ли один класс подклассом другого, то есть наследуется ли он от указанного класса

Особенности issubclass



Функция возвращает `True`, если класс **наследуется от другого класса напрямую или через цепочку**, и `False` — если нет.

Функция `issubclass`



Синтаксис

```
issubclass(class_a, class_b)
issubclass(class_a, tuple_of_classes)
```

Пояснение

- `class_a` — класс, который мы проверяем
- `class_b` — предполагаемый родительский класс
- `tuple_of_classes` — кортеж из нескольких классов

Пример

```
class Employee:
    pass

class Programmer(Employee):
    pass

class Manager(Employee):
    pass

class BackendDeveloper(Programmer):
    pass

print(issubclass(Programmer, Employee)) # Прямой потомок
print(issubclass(BackendDeveloper, Programmer)) # Прямой потомок
print(issubclass(BackendDeveloper, Employee)) # Потомок через цепочку
print(issubclass(Manager, Programmer)) # Разные ветки иерархии
print(issubclass(Employee, object)) # Все классы наследуют от object
```



ВОПРОСЫ





ЗАДАНИЯ





Выберите верные варианты ответа

1. Что вернёт функция `issubclass()` и почему?

```
class Book:
    pass
```

```
b = Book()
print(issubclass(b, object))
```



Выберите верные варианты ответа

1. Что вернёт функция `issubclass()` и почему?

```
class Book:
    pass
```

```
b = Book()
print(issubclass(b, object))
```

Ответ: `TypeError`, так как функция `issubclass()` принимает класс, а не объект.



Выберите верные варианты ответа

2. Для чего используется функция isinstance?

- a. Проверяет, является ли объект экземпляром заданного класса или его наследников
- b. Проверяет, что класс унаследован от object
- c. Проверяет тип переменной только для встроенных типов



Выберите верные варианты ответа

2. Для чего используется функция isinstance?

- a. Проверяет, является ли объект экземпляром заданного класса или его наследников
- b. Проверяет, что класс унаследован от object
- c. Проверяет тип переменной только для встроенных типов

Ответ: а



ВОПРОСЫ





Наследование от object



Класс object

Это корневой класс всей иерархии классов в Python. Он предоставляет набор базовых методов, таких как `__str__()`, `__init__()` и другие.

Класс object



Пример

```
class Book:
    pass
```

Пояснение

Python воспринимает это так:

```
class Book(object):
    pass
```


Существующие в классе методы



Пример

```
print(dir(Book))
```

Пояснение

Список уже существующих в классе методов можно увидеть с помощью функции `dir()`

Магический метод `__str__`



Пример

```
b = Book()

print(b.__str__()) # Вызов метода
__str__ напрямую
print(b)           # Вызов метода
__str__
```

Пояснение

Когда объект выводится через `print()`, Python вызывает магический метод `__str__()`, чтобы получить **строковое представление объекта**, которое будет показано пользователю

Особенности метода `__str__`



Если метод `__str__()` не определён, Python выводит **техническую информацию**



Определение метода `__str__()` позволяет **вернуть читаемую строку** о содержимом объекта



Не должен ничего выводить сам — **только возвращать строку**



Вызывается автоматически при `print(obj)` или `str(obj)`

Пример: техническая информация

```
class Book:  
    def __init__(self, title, author):  
        self.title = title  
        self.author = author
```

```
book = Book("1984", "George Orwell")  
print(book)
```

Пример: содержимое объекта

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"{self.title} by {self.author}"

book = Book("1984", "George Orwell")
print(book)           # вызывает __str__
print(str(book))      # вызывает __str__
print(book.__str__()) # вызывает __str__
```

Преимущества наследования от object



Все классы получают **единое поведение по умолчанию**



Все объекты можно безопасно использовать с базовыми функциями и методами



Это делает классы совместимыми со встроенными механизмами Python (например, `print()` вызывает `__str__()` из `object`, если не переопределено)



ВОПРОСЫ

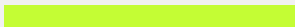




ПРАКТИЧЕСКАЯ РАБОТА



1. Класс Employee



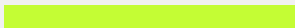
Создайте класс Employee, представляющий сотрудника.

- У каждого объекта должно быть поле name.
- Метод work() выводит строку: <имя> is working....
- Проверьте работу класса, создав сотрудника и вызвав метод work().

Пример вывода:

Alice is working...

2. Класс Developer



Создайте класс `Developer`, который расширяет `Employee`.

- Добавьте дополнительное поле `language`.
- Переопределите метод `work()`, чтобы он включал сообщение из родительского метода и добавлял строку:
`<имя> writes <язык> code.`
- Проверьте работу, создав объект `Developer` и вызвав метод `work()`.

Пример вывода:

```
Bob is working...
Bob writes Python code.
```



ДОМАШНЕЕ ЗАДАНИЕ



Домашнее задание

1. Класс Person

Создайте класс Person, представляющий человека.

- Каждый человек должен иметь имя.
- Добавьте метод `introduce()`, который выводит приветствие с именем.

Пример вывода:

Hello, my name `is` Alice.

Домашнее задание

2. Класс Student

На основе класса `Person` создайте класс `Student`.

- Студент должен иметь имя и номер курса.
- Метод `introduce()` должен сначала выводить базовое приветствие, а затем строку: `I'm on course <номер_курса>`.

Пример вывода:

```
Hello, my name is Alice.
```

```
I'm on course 2.
```

Домашнее задание

3. Класс Teacher и список людей

На основе класса Person создайте класс Teacher.

- У преподавателя есть имя и предмет.
- Метод `introduce()` должен выводить имя и предмет.
- Метод `introduce()` должен выводить строку: `Hello, I am professor <имя>. My subject is <предмет>.`
- Создайте список, в котором будут `Student` и `Teacher`, и вызовите у всех метод `introduce()`.

Пример вывода:

```
Hello, my name is Alice.
I'm on course 2.
Hello, I am professor Bob.
My subject is Mathematics
```

Заключение

